

实验零：实验环境的搭建

实验日期：2025. 10. 17

姓名：何宗霖

学号：23374275

班级：231516

任课教师：李家军

一、实验目的

1. 掌握 Verilog-HDL 编程环境的搭建，包括安装和配置 VSCode、Verilog-HDL 插件、Universal Ctags 和 iverilog 等工具；
2. 通过软件仿真环境的配置和测试 4 位乘法器，学习如何观察仿真波形，验证仿真平台的有效性；
3. 通过使用 Quartus II 软件进行编译、下载到实验平台的操作，熟悉 FPGA 开发的全流程，包括设计思想、软件环境参数配置、仿真、管脚分配和下载等关键步骤，进而熟悉全套开发流程。

二、实验原理

1. FPGA 与开发套件

FPGA (Field Programmable Gate Array, 即现场可编程门阵列) 它是在 PAL、GAL、CPLD 等可编程器件的基础上进一步发展的产物。它是作为专用集成电路 (ASIC) 领域中的一种半定制电路而出现的，既解决了定制电路的不足，又克服了原有可编程器件门电路个数有限的缺点。

FPGA 采用“查找表+编程寄存器”等结构形式构成阵列。ALTERA 公司 Cyclone 器件主要由逻辑阵列块(LAB)、嵌入式存储器块、I/O 单元和 PLL 等模块构成，在各个模块之间存在着丰富的互连线和时钟网络。

Cyclone 器件的可编程资源主要来自逻辑阵列块(LAB)，而每个 LAB 都是由多个逻辑单元 (LE) 来构成，后者是 Cyclone FPGA 器件的最基本可编程单元。Logic Elements(LEs)逻辑单元包括查找表 LUT、可编程的寄存器、级联和进位扩展电路、可编程控制电路、局部和全局的内部连接总线。

查找表 LUT 用来产生组合逻辑，它能产生 4 输入变量的逻辑函数。当需要产生更多的输入变量的逻辑函数时，级联链这样的高速数据通道可以把相邻的查找表 LUT 并联起来，实现其逻辑功能要求。

FPGA 既解决了定制电路的不足，又克服了原有可编程器件门电路数有限的缺点。可以认为 FPGA 能完成任何数字器件的功能，上至高性能 CPU，下至简单的 74、40 系列等集成电路。设计者可以自由地设计一个数字系统，对设计方案进行时序仿真和功能仿真，验证设计的正确性。在完成印刷电路板 PCB 后，还可以利用 FPGA 的在线修改能力，随时随地对功能软件进行修改设计而不必改动硬件电路。

2. FPGA 开发全流程

流程：

代码编写 (coding) → 功能仿真 (function simulation) → 综合 (synthesis) → 综合后仿真 (post-synthesis simulation) → 映射和布线 (map&route) → 后仿真 (timing simulation) → 下载验证 (verification)

电路设计与输入：设计输入方法由硬件描述语言和原理图设计输入方法。原理图设计输入缺点是，当所用芯片升级换代后，改动量过大。因此推荐使用硬件描述语言。

功能仿真：为 RTL 行为级仿真，在大部分设计中执行的第一个仿真将 RTL 行为级仿真。这个阶段的仿真可以用来检查代码中的语法错误以及代码行为的正确性，不考虑竞争冒险，门电路延迟信息。绝大部分设计人员将这个阶段的仿真叫功能仿真。功能仿真也被称为前仿真，通过仿真可以发现设计中的错误，加快设计的进度，提高设计的可靠性。

综合：将设计输入翻译成门、触发器等基本逻辑单元组成的网表，供布局布线器使用。

综合后仿真：综合完成后要检查综合结果是否与原设计一致。可估计门级延时带来的影响，不能估计线延时。一般在设计流程中的第二个仿真是综合后门级功能仿真。绝大多数的综合工具除了可以输出一个标准网表文件以外，还可以输出 Verilog 或者 VHDL 网表，其中标准网表文件是用来在各个工具之间传递设计数据的，并不能用来做仿真使用，而输出的 Verilog 或者 VHDL 网表可以用来仿真。之所以叫门级仿真是因为综合工具给出的仿真网表已经是与生产厂家的器件的底层元件模型对应起来了，所以为了进行综合后仿真必须在仿真过程中加入厂家的器件库，对仿真器进行一些必要的配置，不然仿真器并不认识其中的底层元件，得不到元件的延时信息，无法进行仿真。

映射和布线：映射和布线是将综合的结果适配到 PLD 内部固有的硬件结构上。一般采用时序驱动引擎进行布局布线，考虑面积和速度两个因素。

时序仿真和验证：将布局 and 布线的时延信息反馈注入到设计网表中，所进行的仿真叫时序仿真或后仿真。

下载验证：使用专用电缆，将目标代码下载到配置芯片或者 PLD 内部的存储器，验证设计是否正确。

三、实验内容

1. 安装 vscode 软件；
2. 安装 Verilog-HDL 插件；
3. 安装 Universal Ctags 并设置到系统路径；
4. 安装 iverilog (内含 gtkwave)；
5. 插件设置与四位乘法器环境测试，观察波形图；
6. 使用 QuartusII 软件进行编译、烧录到实验平台上验证。

四、硬件资源

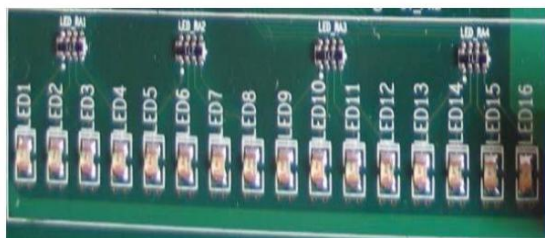
1. 实验用 FPGA 芯片:在本实验中，使用了 Altera (Intel) 公司 Cyclone IV 系列 FPGA 芯片，型号为 EP4CE10F17C8N，有近 10k 逻辑资源，速度等级为 8，FBGA 封装，256 个 IO 引脚。

资源	EP4CE6	EP4CE10	EP4CE15	EP4CE22	EP4CE30	EP4CE40	EP4CE55	EP4CE75	EP4CE115
逻辑单元 (LE)	6, 272	10, 320	15, 408	22, 320	28, 848	39, 600	55, 856	75, 408	114, 480
嵌入式存储器 (Kbits)	270	414	504	594	594	1, 134	2, 340	2, 745	3, 888
嵌入式 18 × 18 乘法器	15	23	56	66	66	116	154	200	266
通用 PLL	2	2	4	4	4	4	4	4	4
全局时钟网络	10	10	20	20	20	20	20	20	20
用户 I/O 块	8	8	8	8	8	8	8	8	8
最大用户 I/O (注释 1)	179	179	343	153	532	532	374	426	528

【注】：EP4CE10F17C8N 为 PPT 上所写，实际 Quartus 用的是 EP4CE115F23C7 型号的芯片，PPT 中并未作出介绍

2. LED 指示灯：

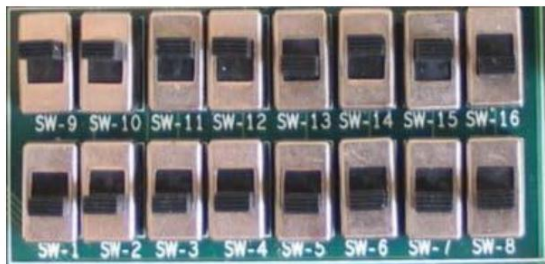
开发平台上有 2x8 共 16 个 LED 指示灯，便于对用户的操作给出提示，使用户得到相应的反馈信息，也可验证设计的正确性。注意为低电平时亮。



LED1	0V_D5	由开关 VLP0 选择	CON1. 2	C10
LED2	0V_D6	由开关 VLP0 选择	CON1. 3	E9
LED3	0V_D7	由开关 VLP0 选择	CON1. 4	D10
LED4	0V_D8	由开关 VLP0 选择	CON1. 5	V13
LED5	0V_D9	由开关 VLP0 选择	CON1. 6	V14
LED6	VSYN	由开关 VLP0 选择	CON1. 7	Y17
LED7	PCLK	由开关 VLP0 选择	CON1. 8	W17
LED8	0V_RES	由开关 VLP0 选择	CON1. 9	W19

3. 拨码开关：

开发平台上有二组 16 只拨码开关输入，开关拨置上为逻辑“1”电平，开关拨置下为逻辑“0”电平。



SW1			CON1. 33	M16
SW2			CON1. 34	R19
SW3			CON1. 54	AA14
SW4			CON1. 61	AA1
SW5			CON2. 62	A3
SW6			CON2. 61	B4
SW7			CON2. 60	C4
SW8			CON2. 59	E6

4. 对应关系：

在本次实验中，clk_sys 对应 50Mhz 时钟。LED1-8 分别对应 out[7]-[10]。SW1-4 对应 x[3]-[0], SW5-8 对应 y[3]-[0]。也即输出端上接 LED 指示灯，使用 LED1-8 指示八位输出情况。输入端上接拨码开关，使用 SW1-4、SW5-8 分别表示两个四位输入。

五、实验过程与结果

1.安装 vscode、Verilog-HDL 插件、Universal Ctags、iverilog;

2.插件设置与环境测试——测试 4 位乘法器

1) 撰写代码

【注】：本人在自学阶段，研究了 multiplier 硬件思路背后的 Booth 算法，并于代码中做了注释。同时手算验证了算法正确性

multiplier4bit.v:

```
multiplier4bit.v
D: > VLSI-project > multiplier4bit > multiplier4bit.v
1 module multiplier4bit(x, y, out, fin, clk_sys);
2
3     input [3:0] x, y;
4     input clk_sys; //时钟信号
5     output reg [7:0] out;
6     output wire fin; //乘法完成信号
7
8     reg [2:0] count; //3位寄存器, 用于计数, 最多4次, 不能两位是因为要表示0-4共5个状态 (0表示结束, 4到1表示4次计算)
9
10    always @(negedge clk_sys) begin
11        if (count==4&count>0) begin
12            if (count==4) out = {4'b0000, y}; //count=4代表刚开始(count初始化为4), 初始化out, 把y放到out的低4位, 高4位初始化为0
13            out = {out[0], out[7:1]}; //最低位放到最高位, 其他位右移一位 (等价于booth是算法里的左移)
14            if (out[7]) out[7:3] = out[6:3] + x; //这里左边5位, 右边4位是要考虑进位 (不进位最高位就补0)
15        end
16    end
17
18    assign fin = (count==0) ? 1 : 0; //count=0, 此时fin=1, 代表乘法完成, count=0立马结束, 不用再左移了 (做题易错)
19
20    reg [3:0] x_last, y_last; //两个4位寄存器, 用于保存上一个周期的 x 和 y 值。
21    always @(negedge clk_sys) begin
22        if ((x_last==x)&(y_last==y)) begin
23            if (count>0) count <= count - 1;
24        end
25        else begin
26            count <= 4;
27            x_last <= x;
28            y_last <= y;
29        end
30    end
31
32 endmodule
```

multiplier4bit_tb.v:

```
multiplier4bit_tb.v
D: > VLSI-project > multiplier4bit > multiplier4bit_tb.v
1 `timescale 1ns/1ns // 时间单位/精度
2
3 module multiplier4bit_tb();
4
5     reg [3:0] x, y;
6     wire [7:0] out;
7     wire fin;
8
9     reg clk_sys;
10    parameter ClockPeriod = 10; //自定义时钟周期
11
12    always #(ClockPeriod/2) clk_sys = ~clk_sys;
13
14    initial begin
15        $dumpfile("wave.vcd"); // 指定VCD文件的名字为wave.vcd, 仿真信息将记录到此文件
16        $dumpvars(0, multiplier4bit_tb); // 指定层次数为0, 则multiplier4bit_tb模块及其下面各层次的所有信号将被记录
17        clk_sys = 1'b0;
18
19        x = 4'b0001; y = 4'b0001; //1*1=1
20        #100
21        x = 4'b0011; y = 4'b0001; //3*1=3
22        #100
23        x = 4'b1011; y = 4'b1001; //11*9=99
24        #100
25        x = 4'b1111; y = 4'b1111; //15*15=225
26        #100
27        $finish;
28    end
29
30
31    multiplier4bit multiplier4bit_obj(x, y, out, fin, clk_sys);
32
33 endmodule
34
35 //iverilog -o multiplier4bit.out multiplier4bit.v multiplier4bit_tb.v
36 //vvp multiplier4bit.out
37 //gtkwave wave.vcd
```

手算验证:

刷题 3-第三章: 遥... Chapter_3_时... 自动控制原理

$10 \times 6 = 60$

1010.
0110

count = 0

0000 0110 初始化

0000 0011 左移

$x[7] \neq 1$

count = 3

1000 0001 左移

$x[7] = 1$

$0000 + 1010 = 1010$

$\Rightarrow$ 0101 0001

count = 2

1010 1000 左移

$x[7] = 1$

$0101 + 1010 = 1111$

$\Rightarrow$ 0111 1000

$32 + 16 + 12$

count = 1

0011 1100 左移

$x[7] \neq 1$

count = 0 不移了.

$16 + 14 = 30$

1110 0001

2) 编译文件

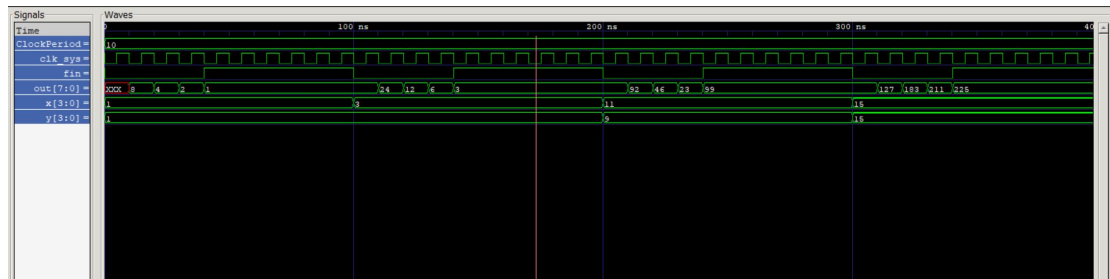
命令行: iverilog -o multiplier4bit.out multiplier4bit.v multiplier4bit_tb.v

3) 生成 .vcd 波形文件

命令行: vvp multiplier4bit.out

4) 观察波形

命令行: gtkwave wave.vcd



3. 下载验证, 将 4 位乘法器下载到试验箱, 并观察现象。

1) 打开 Quartus II, 创建新工程, 选择芯片家族类型 Cyclone IV E, 选择具体型号 EP4CE115F23C7。

【注】如果此处登录闪退, 有以下改进方法:

首先破译的网网卡 ID 选择以太网的网卡

(就算这个网课下面可能显示不可操作)

其次尝试关闭防火墙进入, 如果后续运行报错也可能是因为这个

最后尝试使用管理员身份运行程序 (本人最终是这么解决的)

2) 新建文件, 选择 verilog 类型, 由于试验箱 LED 为低使能, 将输出文件临时取反, 修改代码如下:

```
multiplier4bit.v  multiplier4bit_tb.v  module multiplier4bit2.py 9+ x
D: > VLSI-project > multiplier4bit > module multiplier4bit2.py
1  module multiplier4bit(x, y, out, fin, clk_sys); //修改
2
3  input [3:0] x, y;
4  input clk_sys;
5  output wire [7:0] out;
6  reg [7:0] res;
7
8  reg [2:0] count;
9
10 always @(negedge clk_sys) begin
11     if (count <= 4 & count > 0) begin
12         if (count == 4) res = {4'b0000, y};
13         res = {res[0], res[7:1]};
14         if (res[7]) res[7:3] = res[6:3] + x;
15     end
16 end
17
18 reg [3:0] x_last, y_last;
19 always @(negedge clk_sys) begin
20     if ((x_last == x) & (y_last == y)) begin
21         if (count > 0) count <= count - 1;
22     end
23     else begin
24         count <= 4;
25         x_last <= x;
26         y_last <= y;
27     end
28 end
29
30 assign out = ~res;
31
32 endmodule
```


粘贴到 Quartus II 保存，编译。

3) USB-Blaster 驱动安装

4) 绑定管脚

表 1 输入端管脚绑定对应表

拨码开关	输入端量	输入引脚
/	clk_sys	PIN_T1
SW1	x[3]	PIN_M16
SW2	x[2]	PIN_R19
SW3	x[1]	PIN_AA14
SW4	x[0]	PIN_AA1
SW5	y[3]	PIN_A3
SW6	y[2]	PIN_B4
SW7	y[1]	PIN_C4
SW8	y[0]	PIN_E6

表 2 输出端管脚绑定对应表

LED 指示灯	输出端量	输出引脚
LED1	out[7]	PIN_C10
LED2	out[6]	PIN_E9
LED3	out[5]	PIN_D10
LED4	out[4]	PIN_V13
LED5	out[3]	PIN_V14
LED6	out[2]	PIN_V17
LED7	out[1]	PIN_W17

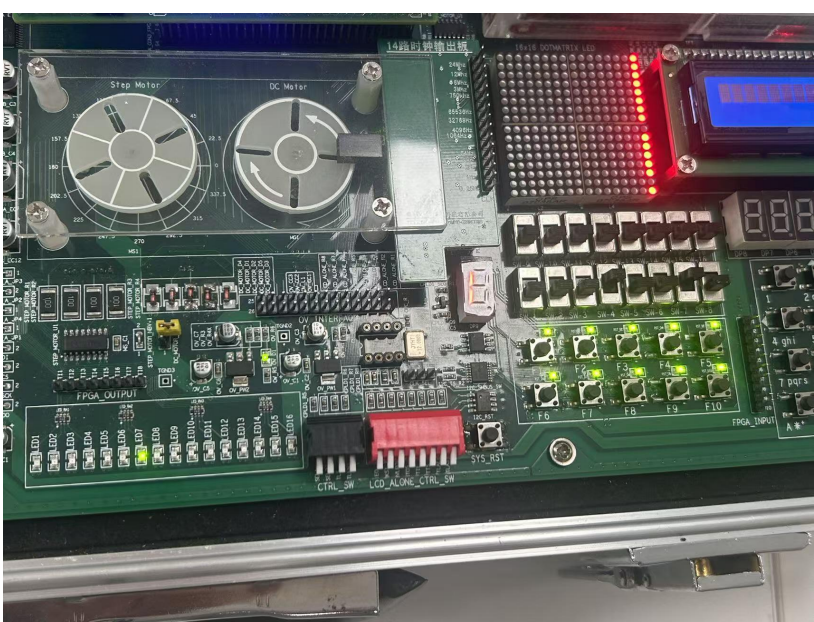
5) 再次编译

6) 选择代烧录的.sof 文件，点击 start 进行烧录

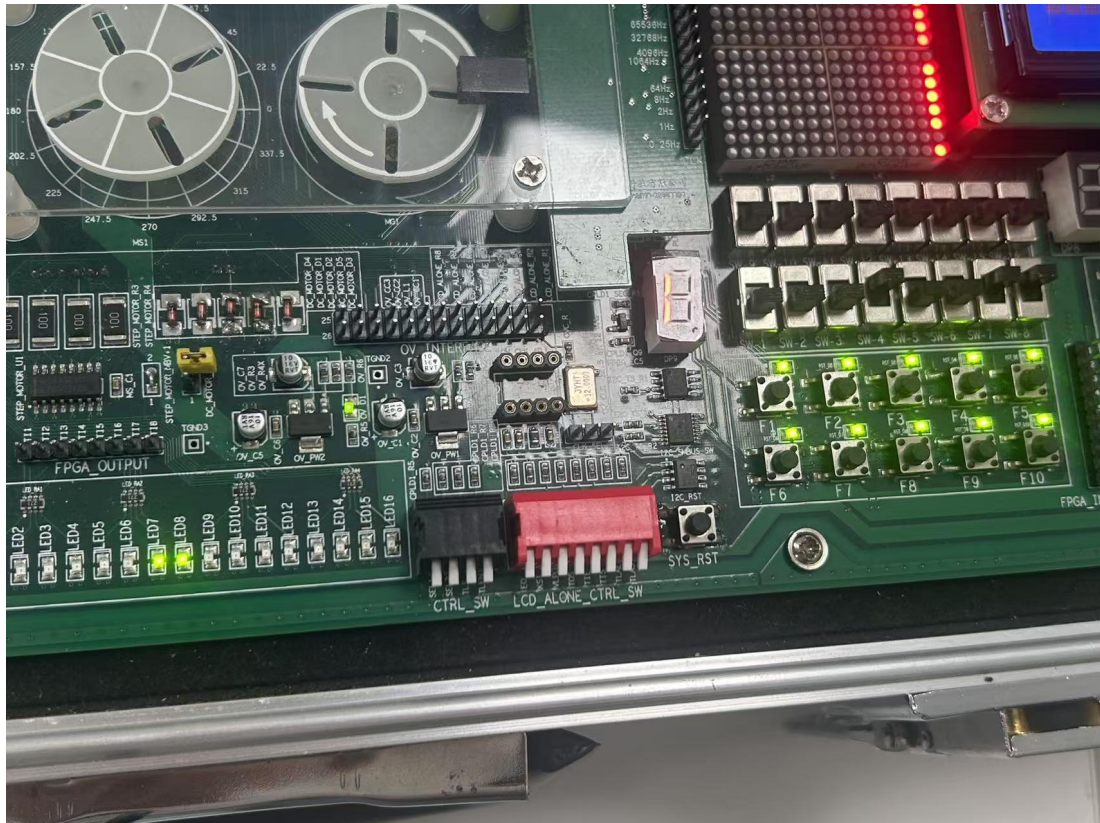
7) 最终结果

第一组：1*2=2

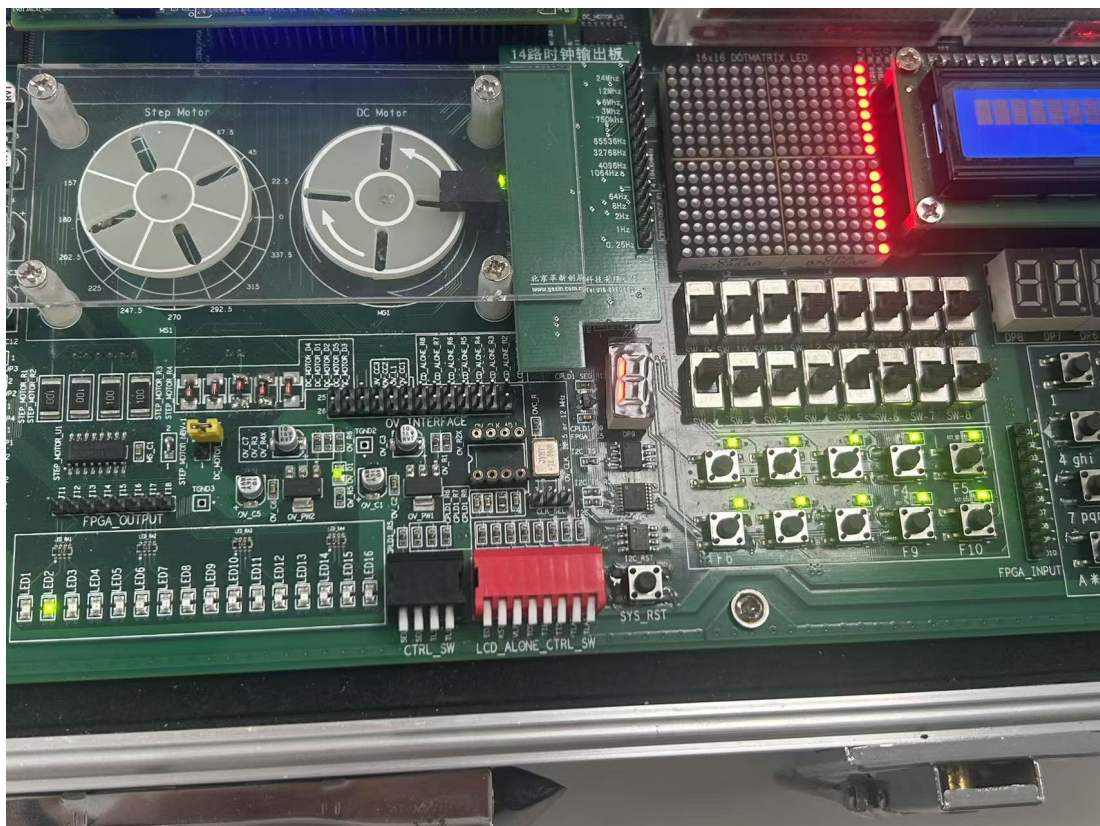
即 0001*0010=0000 0010



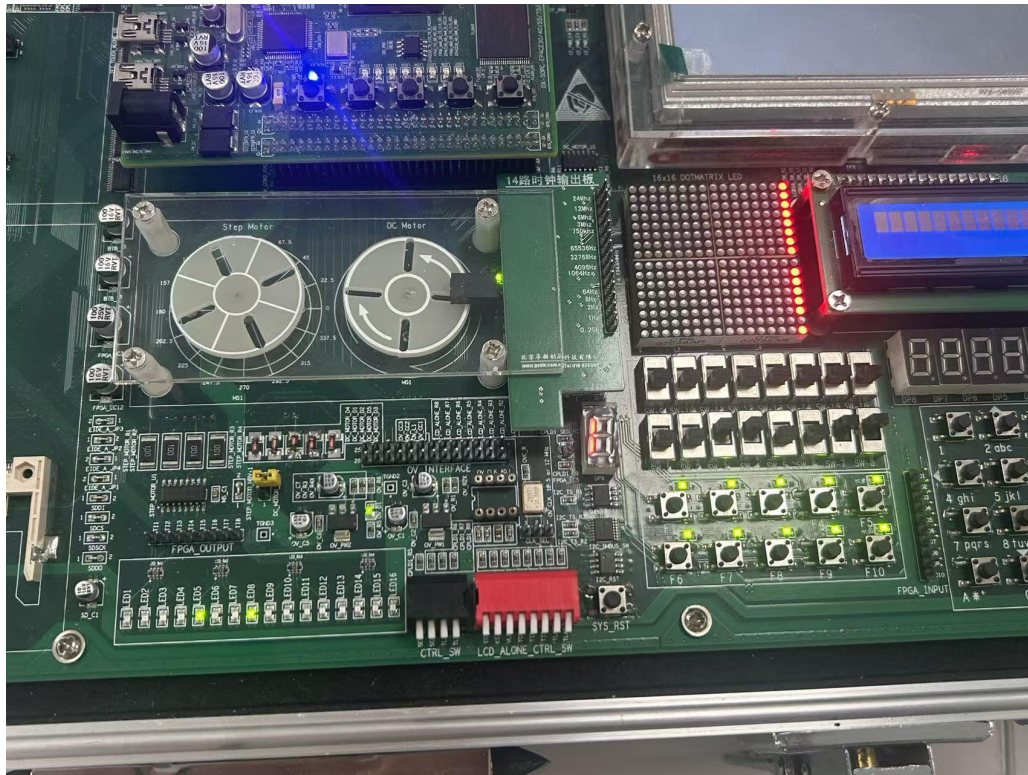
第二组：1*3=3
即 0001*0011=0000 0011



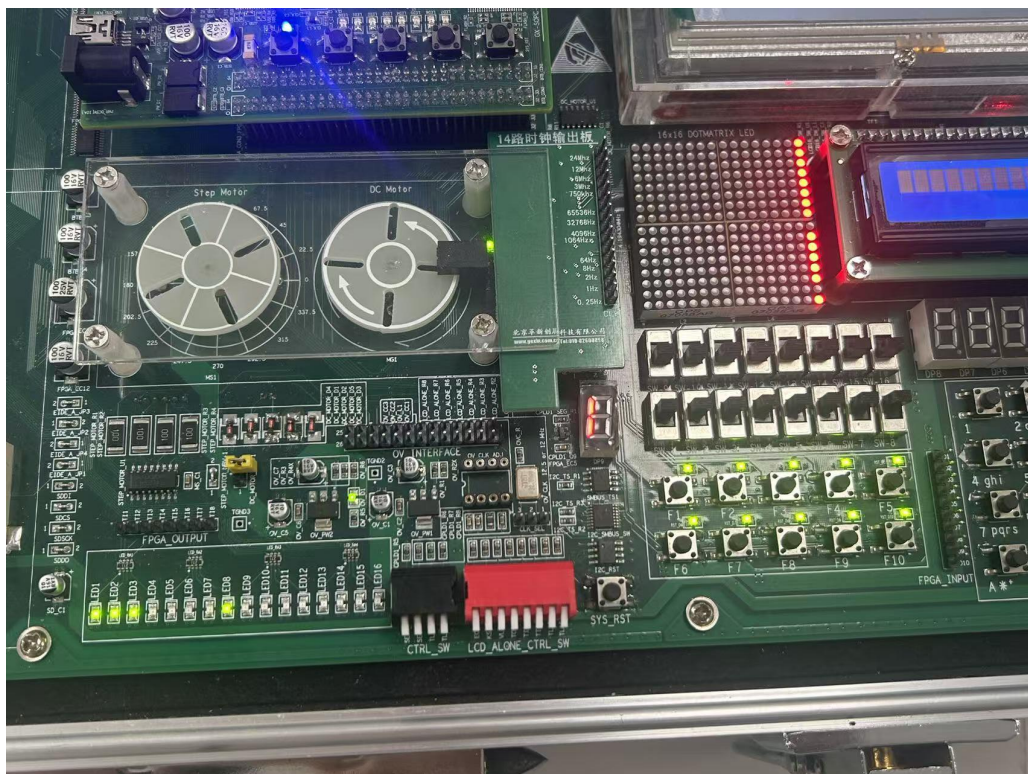
第三组：8*8=64
即：1000*1000=0100 0000



第四组：3*3=9
即：0011*0011=0000 1001



第五组：15*15=225
即：1111*1111=1110 0000



综上，可以得出实验正确。

